

1. Qué endpoints implementaste y qué hace cada uno.

Los endpoints que utilice fueron los siguientes:

- GET /eventos/buscar?q=texto: este lo que hace es que se pueda buscar por nombre
- GET /eventos/nuevo: Esto lo que hace es que se crea un nuevo evento
- GET /eventos/{id}/editar: Permite editar los eventos ya creados
- POST /eventos/{id}: Este lo que hace es que se carguen los datos del evento después de editarlos
- POST /eventos/{id}/eliminar: Sirve para eliminar eventos

2. Por qué elegiste las anotaciones de validación que pusiste en `Evento`.

- Porque son las que se usan generalmente

3. Cómo resolviste el modal de confirmación (qué atributos `data-bs-\*` usaste).

- Para el modal de confirmación lo que hice fue utilizar Bootstrap ya que no se podía usar el confirm() del navegador

SOLUCIÓN - Caso Práctico 1

Endpoints implementados

Durante el desarrollo del proyecto se completaron los endpoints necesarios para terminar el CRUD de eventos.

Se implementó el endpoint **GET /eventos/buscar?q=texto**, el cual permite buscar eventos por su nombre utilizando un parámetro recibido por la URL. El resultado se muestra reutilizando la misma vista del listado de eventos, por lo que no fue necesario crear una página adicional.

También se agregó el endpoint **GET /eventos/nuevo**, cuya función es mostrar un formulario vacío para registrar un nuevo evento.

Para guardar la información del formulario se implementó **POST /eventos**, donde se reciben los datos del evento, se validan y, si todo es correcto, se almacenan en la base de datos. Si existe algún error de validación, el usuario permanece en el formulario para corregir la información.

Para la edición de eventos se implementó **GET /eventos/{id}/editar**, que carga el formulario con los datos del evento seleccionado para que puedan modificarse.

Posteriormente se implementó **POST /eventos/{id}**, encargado de actualizar la información del evento existente después de validar nuevamente los datos ingresados.

Finalmente se agregó **POST /eventos/{id}/eliminar**, el cual elimina el evento seleccionado y redirige nuevamente al listado de eventos.

Validaciones utilizadas

Las validaciones se agregaron utilizando Bean Validation para asegurar que la información almacenada sea válida.

En el nombre del evento se utilizó **@NotBlank** para impedir que quede vacío y **@Size** para limitar la cantidad de caracteres permitidos.

En la descripción, el lugar y otros campos de texto se utilizó **@Size**, evitando que el usuario ingrese textos demasiado largos.

Para la fecha del evento se utilizó **@Future**, ya que un evento no debería registrarse con una fecha pasada.

En el cupo máximo se utilizó **@Positive**, garantizando que siempre exista al menos un espacio disponible para el evento.

En los cupos vendidos también se utilizó una validación numérica para evitar valores inválidos.

En el precio se utilizó `**@PositiveOrZero**`, permitiendo registrar eventos gratuitos, pero evitando valores negativos.

Estas validaciones ayudan a mantener la integridad de los datos antes de guardarlos en la base de datos.

Modal de confirmación

Para eliminar un evento se implementó un modal utilizando Bootstrap en lugar del ``confirm()`` del navegador, tal como lo solicitaba el caso práctico.

Cada botón "Eliminar" contiene los atributos ``data-bs-toggle="modal"`` y ``data-bs-target``, que permiten abrir el modal correspondiente.

Además, mediante atributos personalizados se envía el identificador y el nombre del evento seleccionado para que el modal muestre el nombre del evento que será eliminado y configure correctamente la acción del formulario antes de enviar la solicitud.

De esta forma el usuario puede confirmar o cancelar la eliminación antes de que se ejecute.

Decisiones técnicas

Durante el desarrollo se mantuvo la estructura MVC proporcionada en el proyecto base. El controlador únicamente recibe las solicitudes del usuario y delega la lógica al servicio, mientras que el acceso a la base de datos se realiza únicamente mediante el repositorio.

Se utilizó el tipo `LocalDate` para manejar la fecha del evento, ya que únicamente era necesario almacenar la fecha sin incluir la hora.

Para la búsqueda por nombre se aprovechó un método de consulta de Spring Data JPA utilizando `findByNombreContainingIgnoreCase`, permitiendo realizar búsquedas parciales sin diferenciar entre mayúsculas y minúsculas.

Finalmente, para los formularios de creación y edición se reutilizó la misma vista (`eventos/form.html`), diferenciando automáticamente si el formulario corresponde a un nuevo evento o a la actualización de uno existente según el identificador del objeto recibido.